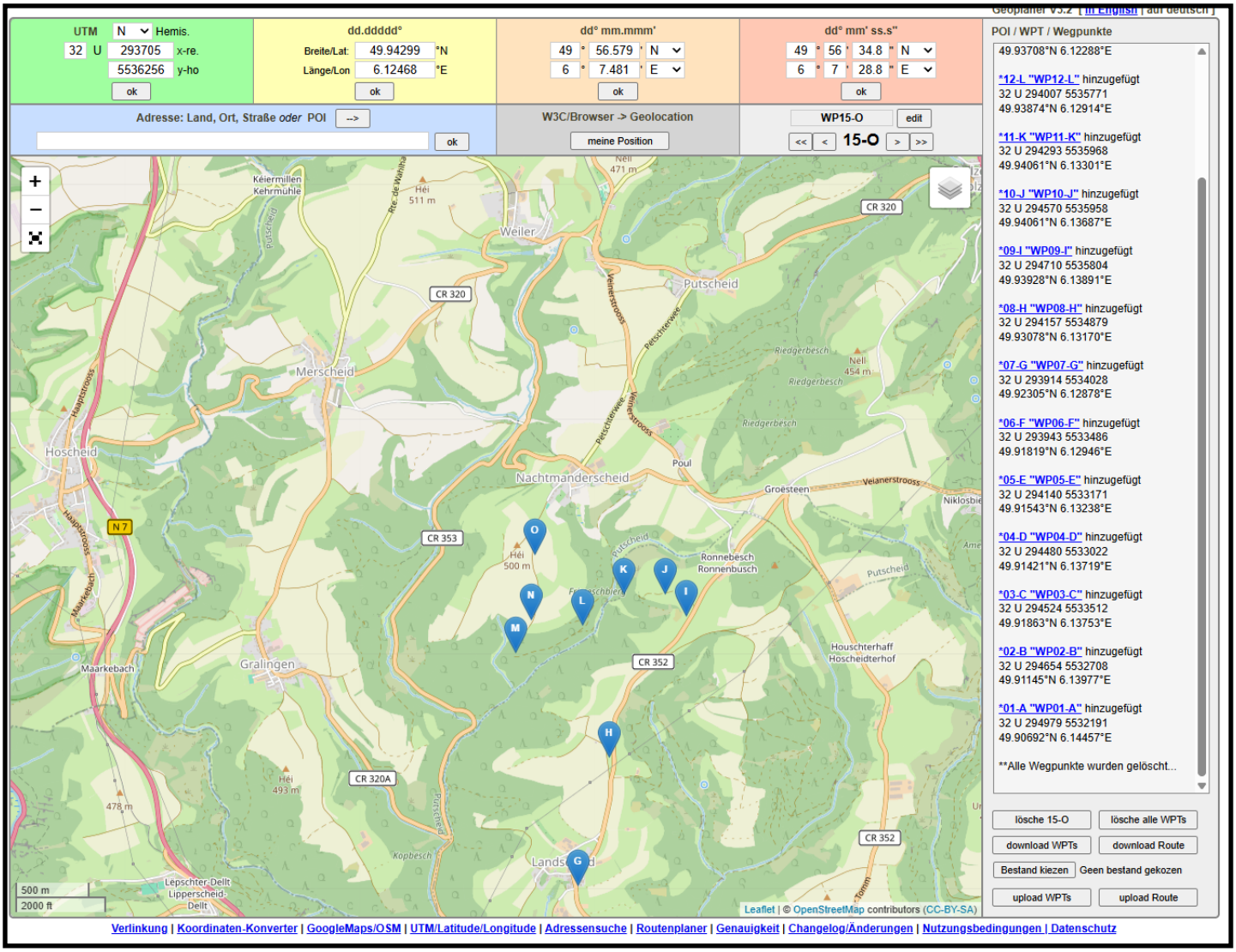


Maak op https://geoplaner.de een aantal waypoints aan zodat die aan de rechtkant genoteerd worden.

De nieuwst aangemaakte waypoint komt bovenaan en daar houdt het scritp rekening mee.

Kopieer de waypoints uit de lijst aan de rechterkant naar een nieuwe (of bestaande) file Waypoints_<callsign>.txt

Sla het bestand Waypoints.txt op in dezelfde directory als het python script.

```
*14-N "WP14-N" hinzugefügt
32 U 293664 5535821
49.93907°N 6.12433°E

*13-M "WP13-M" hinzugefügt
32 U 293551 5535604
49.93708°N 6.12288°E

*12-L "WP12-L" hinzugefügt
32 U 294007 5535771
49.93874°N 6.12914°E

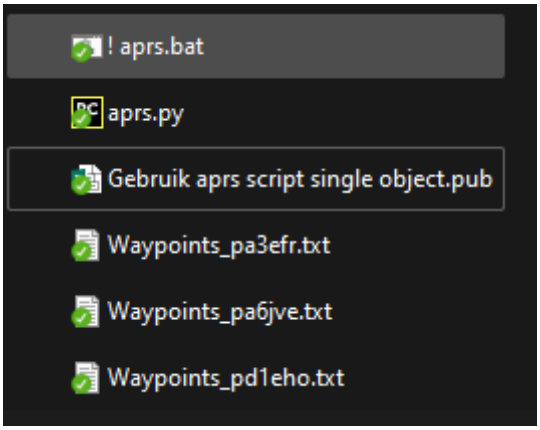
*11-K "WP11-K" hinzugefügt
32 U 294293 5535968
49.94061°N 6.13301°E

*10-J "WP10-J" hinzugefügt
32 U 294570 5535958
49.94061°N 6.13687°E

*09-I "WP09-I" hinzugefügt
32 U 294710 5535804
49.93928°N 6.13891°E

*08-H "WP08-H" hinzugefügt
32 U 294157 5534879
49.93078°N 6.13170°E

*07-G "WP07-G" hinzugefügt
32 U 293914 5534028
49.92305°N 6.12878°E
```



In de directory van het script start !aprs.bat. Dit script laadt automatisch de Waypoints_<callsign>.txt.

Voor meerdere gelijktijdige objecten heb je dus meerdere Waypoints_<callsign>.txt files nodig.

Het script vraag naast het aantal objecten, per objectom een aantal zaken:

1. Te gebruiken callsign met SSID
2. Passcode (zie https://www.iz3mez.it/aprs-passcode/, maar vul de call in zonder SSID) als deze niet al in de APRS_Codes.txt voorkomt
3. Type vervoer bepaalt de snelheid van verplaatsen
    1. SPEED_RANGES = {'auto': (50, 80), 'fiets': (10, 25), 'lopen': (1, 5)}
4. Interval van de te plotten posities
5. Comment dat bij de plot getoond wordt
6. Waypoint bestand voor de callsign met default Waypoints_<callsign>.txt

```
============================================================
APRS Tracker (multi-object)
============================================================
Hoeveel objecten wil je volgen? 3

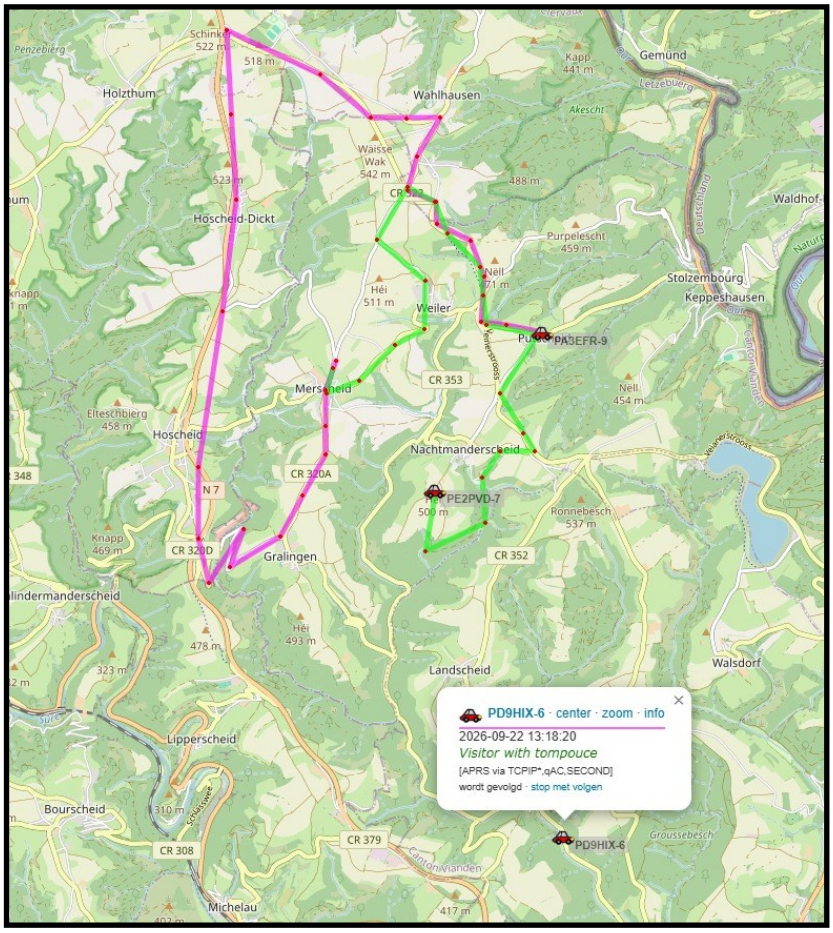
============================================================
Object 1/3
============================================================
Callsign: pa3efr-1
Passcode: 22196
1=Auto 2=Fiets 3=Lopen
Keuze: 3
Interval (s): 22
Comment: Testing

3 waypoints geladen uit Waypoints_pa3efr.txt:
  1: 50.432100,-0.030760
  2: 50.433850,0.036560
  3: 50.436480,0.107990

============================================================
Object 2/3
============================================================
Callsign:
```

In dit voorbeeld is het aprs.py script gestart met elk hun eigen Waypoints_<callsign>.txt.

Daarvoor zijn wel alle passcodes nodig en kun je elk object voorzien van een eigen comment.



Het script

```
#!/usr/bin/env python3
"""APRS Tracker - CORRECTE NOTATIE (multi-object, gelijktijdig)"""

import aprslib
import time
import random
import math
import re
import os
import threading


SPEED_RANGES = {
    'auto': (50, 80),
    'fiets': (10, 25),
    'lopen': (1, 5)
}

WAYPOINTS_FILE_TEMPLATE = "Waypoints_{call}.txt"
APRS_CODES_FILE = "APRS_Codes.txt"


# Herkent regels zoals:
# 49.98172°N 6.08968°E
# Ook regels met een expliciet minteken voor het getal,
# bv: 50.39012°N -0.02114°E
COORD_RE = re.compile(
    r'(-?[\d.]+)\s*°?\s*([NS])\s+(-?[\d.]+)\s*°?\s*([EW])'
)


# Lock zodat de console-output van meerdere objecten niet door elkaar loopt
PRINT_LOCK = threading.Lock()


def safe_print(*args, **kwargs):
```

```
    with PRINT_LOCK:
        print(*args, **kwargs)


def read_aprs_codes(path=APRS_CODES_FILE):
    """
    Leest APRS passcodes uit APRS_Codes.txt.

    Verwacht regels zoals:
        pa3efr 22196
        pd1eho 23212

    De callsign wordt hoofdletterongevoelig verwerkt.
    Een eventuele SSID wordt genegeerd.

    Regels die leeg zijn of met '#' beginnen worden genegeerd.
    """
    codes = {}

    if not os.path.isfile(path):
        return codes

    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.strip()

            if not line or line.startswith("#"):
                continue

            parts = line.split()

            if len(parts) < 2:
                continue

            callsign = parts[0].upper().split("-")[0]
            code = parts[1].strip()

            codes[callsign] = code

    return codes


def get_aprs_passcode(call):
    """
    Bepaal de APRS passcode voor een callsign.

    Eerst wordt gezocht in APRS_Codes.txt met de callsign
    zonder SSID.

    Bijvoorbeeld:
        PA3EFR
        PA3EFR-1
        PA3EFR-9

    zoeken allemaal naar:
        PA3EFR

    Alleen wanneer de callsign niet in APRS_Codes.txt
    voorkomt, wordt de gebruiker om een passcode gevraagd.
    """
    base_call = call.split("-")[0].upper()

    codes = read_aprs_codes()

    if base_call in codes:
        print(
            f"Passcode voor {call} gevonden in "
            f"{APRS_CODES_FILE}: {codes[base_call]}.\n"
        )
        return codes[base_call]
```

```
    print(
        f"Callsign {base_call} staat niet in "
        f"{APRS_CODES_FILE}."
    )

    return input(f"Go to https://www.iz3mez.it/aprs-passcode/ for passcode. Passcode voor {call}: ").strip()


def read_waypoints(path):
    """
    Leest waypoints uit een bestand met blokken van
    (minstens) 3 regels per punt:

      1: *NN-X "naam" hinzugefügt
      2: UTM-regel (wordt genegeerd)
      3: GPS-positie, bv: 49.98172°N 6.08968°E

    Er kunnen willekeurig veel waypoints in het bestand staan.
    Elke regel die op het coördinatenpatroon matcht wordt als
    waypoint meegenomen, in de volgorde waarin ze in het
    bestand voorkomen.
    """
    if not os.path.isfile(path):
        raise FileNotFoundError(
            f"Waypoints-bestand niet gevonden: {path}"
        )

    wp = []

    with open(path, encoding="utf-8") as f:
        for line in f:
            m = COORD_RE.search(line)

            if not m:
                continue

            # Als het getal al een expliciet minteken heeft,
            # is dat leidend.
            lat = float(m.group(1))

            if lat >= 0 and m.group(2) == "S":
                lat = -lat

            lon = float(m.group(3))

            if lon >= 0 and m.group(4) == "W":
                lon = -lon

            wp.append((lat, lon))

    if len(wp) < 2:
        raise ValueError(
            f"Te weinig waypoints gevonden in {path} "
            f"({len(wp)})"
        )

    # In het bestand staat de laatste waypoint bovenaan,
    # dus omdraaien.
    wp.reverse()

    return wp
```